PRACA LICENCJACKA · RAPORT Z POSTEPU

Projekt i implementacja platformy do zarzadzania usluga VPN

vpnVPN — platforma SaaS do zarzadzania infrastruktura VPN

2024 / 2025

O projekcie

Cel pracy

- Zaprojektowanie i realizacja **platformy SaaS** do zarzadzania usluga VPN
- Centralne zarzadzanie **flota serwerow VPN** w chmurze AWS
- Podejscie **privacy-first** — zero logowania ruchu sieciowego
- Obsluga **wielu protokolow**: WireGuard, OpenVPN, IKEv2
- Model subskrypcyjny z platnoscia online (Stripe)

Zakres

- Aplikacja webowa (panel uzytkownika + admin)
- Backend — control plane + serwis metryk
- Serwer VPN w Rust (agent na maszynach EC2)
- Aplikacja desktopowa — klient VPN (Tauri + Rust daemon)
- Infrastruktura w chmurze AWS (Infrastructure as Code)

Struktura monorepo

```
vpnVPN/
  apps/
    web/           # Next.js 15, panel SaaS
    desktop/       # Tauri + Rust daemon
    vpn-server/    # Rust, agent VPN
  services/
    control-plane/ # Bun/Fastify REST API
    metrics/       # Bun/Fastify, ingestion metryk
  packages/
    db/            # Prisma schema, 14 modeli
  infra/
    pulumi/        # IaC w TypeScript
```

Narzedzia monorepo

`Bun` runtime i package manager, `Turborepo` orkiestracja buildow, `Biome` linting/formatowanie, `GitHub Actions` CI/CD.

Architektura systemu

Frontend

Panel uzytkownika, admin, platnosci, dashboard

`Next.js 15` `Vercel`

Control Plane

REST API, rejestracja serwerow, peery, tokeny

`Bun/Fastify` `Lambda`

VPN Server

Agent na EC2, tunele VPN, sync peerow co 2s

`Rust` `EC2`

Desktop

Klient VPN, daemon root, GUI, IPC

`Tauri` `Rust`

Przeplyw danych

- Uzytkownik rejestruje sie na stronie, oplaca subskrypcje przez **Stripe**
- Frontend tworzy peera w control plane (`POST /peer/add` , API key auth)
- Serwery VPN co 2s pobieraja liste peerow (`GET /server/peers` , Bearer token)
- Kazdy backend wywoluje `apply_peers()` — aktualizuje konfiguracje tuneli
- Desktop laczy sie z wybranym serwerem przez `wg-quick` , `openvpn` lub `ipsec`

Bezpieczenstwo

- **Zero logowania ruchu** — serwery nie zapisuja danych o pakietach
- **Szyfrowanie** — ChaCha20-Poly1305 (WG), AES-256-GCM (OVPN/IKEv2)
- **Auth** — OAuth 2.0 + magic link (web), OTP (desktop)
- **Daemon** — oddzielny proces root, GUI nie ma uprawnien sieciowych
- **PKI** — certyfikaty x509 generowane przez `rcgen` (Rust)

Technologie VPN — przeglad

WireGuard

- Nowoczesny protokol — **~4000 LOC** w jadrze Linuxa
- Warstwa 3 (IP), interfejs **TUN**
- Szyfrowanie: **ChaCha20-Poly1305**
- Wymiana kluczy: **Curve25519** (ECDH)
- Auth: **para kluczy publicznych**
- Najszybszy, najnizszy narzut

Glowny protokol — domyslnie wlaczony

OpenVPN

- Dojrzaly protokol, dziala w **user space**
- Warstwa 3, interfejs **TUN** (tun1)
- Szyfrowanie: **AES-256-GCM**
- Auth TLS: **RSA 2048** (self-signed)
- Auth klienta: **username/password**
- Szeroka kompatybilnosc platform

Drugi protokol — fallback

IKEv2 / IPsec

- Protokol IPsec z **IKEv2** key exchange
- Warstwa kernela, **strongSwan** (serwer)
- Szyfrowanie: **AES-256-GCM**
- Wymiana kluczy: **ECP256** (ECDH)
- Wbudowany w OS (macOS, Windows)
- Szybki **reconnect** — MOBIKE

Trzeci protokol — czesciowy

Wspolna abstrakcja — trait `VpnBackend` (vpn/mod.rs)

Strona serwera (apps/vpn-server/src/vpn/mod.rs)

```
pub trait VpnBackend: Send + Sync {
  fn protocol(&self) -> VpnProtocol;
  fn start(&self) -> Result<()>;
  fn stop(&self) -> Result<()>;
  fn status(&self) -> Result<BackendStatus>;
  fn apply_peers(&self, &[PeerSpec]) -> Result<()>;
  fn public_key(&self) -> Option<String>;
}
```

Strona klienta (apps/desktop/daemon/src/vpn/mod.rs)

```
pub trait VpnBackend: Send + Sync {
  fn connect(&self, config: &VpnConfig)
    -> impl Future<Output = Result<ConnectionStatus>>;
  fn disconnect(&self)
    -> impl Future<Output = Result<()>>;
  fn status(&self)
    -> impl Future<Output = Result<ConnectionStatus>>;
}
```

WireGuard — implementacja

Serwer — `vpn/wireguard.rs`

- `WireGuardBackend::new(port)` — config dir: `/etc/wireguard`
- Klucz prywatny: `Command::new("wg").arg("genkey")` → `server.key`
- Klucz publiczny: `Command::new("wg").arg("pubkey")` (stdin: privkey)
- Interfejs: `ip link add wg0 type wireguard` lub **wireguard-go** fallback
- Konfiguracja: `wg setconf wg0 wg0.conf` + `ip addr add 10.8.0.1/24`
- Start: `ip link set up dev wg0`

apply_peers() — sync co 2s

- Control plane zwraca liste `PeerSpec` z `public_key` + `allowed_ips`
- Buduje nowy `wg0.conf` z sekcjami `[Peer]`
- Zapis do `wg0.conf.new` (atomowy swap)
- Linux/macOS: `wg syncconf wg0 wg0.conf.new` — **live update** bez przerwy
- Fallback: pelny `wg setconf` jesli syncconf zawiedzie
- Windows: podmiana conf + restart serwisu

Serwer — start() (wireguard.rs:151)

```rust
fn start(&self) -> Result<()> {
  self.ensure_keys_and_config()?;
  // wg genkey → /etc/wireguard/server.key
  // wg0.conf: [Interface] PrivateKey + ListenPort

  let nm = get_network_manager();
  nm.configure_wireguard(
    "wg0", "10.8.0.1/24", self.listen_port,
    &self.conf_path()
  )?; // ip link add + wg setconf
  nm.bring_up_interface("wg0")?;
  // ip link set up dev wg0
}
```

Klient — connect() (daemon/vpn/wireguard.rs:61)

```rust
let wg_config = build_config(config)?;
// [Interface] PrivateKey, Address, DNS
// [Peer] PublicKey, AllowedIPs=0.0.0.0/0,::/0
//        Endpoint, PersistentKeepalive=25
fs::write(&config_path, &wg_config)?; // chmod 0o600

// macOS/Linux:
Command::new(&wg_quick).args(["up", path]);
// Windows:
Command::new("wireguard.exe")
  .args(["/installtunnelservice", path]);
// Weryfikacja: wg show → "latest handshake:"
// Rollback jesli brak handshake po 6 probach
```

OpenVPN — implementacja

Serwer — `vpn/openvpn.rs`

- Self-signed cert: `openssl req -x509 -newkey rsa:2048 -sha256 -days 3650`
- Serwer nasuchuje na porcie **1194/UDP** (env `OPENVPN_PORT`)
- Auth klienta: `auth-user-pass-verify verify.sh via-env`
- Skrypt `verify.sh`: `grep -F -x "$username $password" secrets.txt`
- Management interface na `127.0.0.1:7505` do monitoringu
- Status: parsowanie `/var/run/openvpn-status.log` (wersja 3)

apply_peers() — aktualizacja credentials

- Iteruje po `PeerSpec` — filtruje peery z `username` + `password`
- Zapisuje do `/etc/openvpn/secrets.txt`: `"user pass\n"` per linia
- OpenVPN czyta plik przy kazdej probie auth — **nie wymaga reloadu**
- Metryki: `CLIENT_LIST,CN,Addr,BytesRx,BytesTx` z status.log

Konfiguracja serwera: `proto udp`, `dev tun1`, `server 10.9.0.0/24`, `topology subnet`, `verify-client-cert none`, `username-as-common-name`

Serwer — konfiguracja (openvpn.rs:52)

```
// Generowana dynamicznie w ensure_certs_and_conf()
port 1194
proto udp
dev tun1
server 10.9.0.0 255.255.255.0
verify-client-cert none
username-as-common-name
script-security 2
auth-user-pass-verify /etc/openvpn/verify.sh via-env
management 127.0.0.1 7505
status /var/run/openvpn-status.log 5
status-version 3

// Start: openvpn --config server.conf --daemon
// Stop:  pkill openvpn
```

Klient — connect() (daemon/vpn/openvpn.rs:8)

```
fn build_config(cfg: &VpnConfig) -> String {
  "client
dev tun
proto udp
remote {endpoint} {port}
cipher AES-256-GCM
auth SHA256
remote-cert-tls server
keepalive 10 60"
}
// Auth: zapisuje username + password
// do vpnvpn-auth.txt (chmod 0o600)

// Unix:  openvpn --config ... --daemon
//        --log /var/log/vpnvpn-openvpn.log
// Win:   openvpn.exe --config ...
// Stop:  pkill -SIGTERM openvpn (Unix)
//        taskkill /IM openvpn.exe (Win)
```

IKEv2 / IPsec — implementacja

Serwer — `ikev2.rs` + `vpn/ipsec.rs`

- `setup_ikev2()` kopiuje certyfikaty PKI do `/etc/ipsec.d/`
- CA → `/etc/ipsec.d/cacerts/ca.pem`
- Server cert → `/etc/ipsec.d/certs/server.pem`
- Klucz prywatny → `/etc/ipsec.d/private/server.key`
- Generuje `/etc/ipsec.conf` i `/etc/ipsec.secrets`
- Start: `ipsec start --nofork` (spawn jako child process)
- Status: `swanctl -l` → zlicza `ESTABLISHED` / `INSTALLED`

Konfiguracja strongSwan (ikev2.rs:58)

```
conn ikev2-eaptls
  keyexchange=ikev2
  ike=aes256gcm16-prfsha256-ecp256!
  esp=aes256gcm16-prfsha256-ecp256!
  leftcert=server.pem
  leftsendcert=always
  leftsubnet=0.0.0.0/0,::/0
  leftauth=pubkey
  rightauth=pubkey
  rightsourceip=10.9.0.0/24,fd42:42:42::/64
  mobike=yes        # szybki reconnect
  forceencaps=yes   # NAT traversal
  # Secrets: : RSA /etc/ipsec.d/private/server.key
```

Klient — macOS (ikev2.rs:103)

```
// Generuje .mobileconfig (XML plist)
<dict>
  <key>IKEv2</key>
  <dict>
    <key>AuthenticationMethod</key>
    <string>SharedSecret</string>
    <key>RemoteAddress</key>
    <string>{endpoint}</string>
  </dict>
</dict>
// Connect: networksetup
//   -connectpppoeservice "vpnVPN-IKEv2"
// Status: scutil --nc status
```

Klient — Linux (ikev2.rs:253)

```
// strongSwan client config
conn vpnvpn
  keyexchange=ikev2
  type=tunnel
  left=%any
  leftauth=psk
  right={endpoint}
  rightsubnet=0.0.0.0/0
  ike=aes256-sha256-modp2048!
  esp=aes256-sha256!

// ipsec reload → ipsec up vpnvpn
// Status: ipsec status vpnvpn
//   → "ESTABLISHED" = connected
```

Klient — Windows (ikev2.rs:348)

```
// Wbudowany klient Windows
// PowerShell:
Add-VpnConnection
  -Name "vpnVPN-IKEv2"
  -ServerAddress "{endpoint}"
  -TunnelType IKEv2
  -AuthenticationMethod EAP
  -EncryptionLevel Required

// Polaczenie:
rasdial "vpnVPN-IKEv2"
// Disconnect:
rasdial "vpnVPN-IKEv2" /disconnect
```

PKI i kryptografia

Generowanie certyfikatow — `pki.rs`

```rust
use rcgen::{Certificate, CertificateParams,
            IsCa, BasicConstraints, SanType};
use sha2::{Digest, Sha256};

// 1. CA certificate (self-signed)
let ca_params = CertificateParams::default();
ca_params.is_ca = IsCa::Ca(Unconstrained);
ca_params.distinguished_name
  .push(DnType::CommonName, "vpnvpn-ca");
let ca = Certificate::from_params(ca_params)?;

// 2. Server cert (signed by CA)
let srv = CertificateParams::new(vec!["vpnvpn-server"]);
srv.subject_alt_names.push(SanType::IpAddress(ip));
let pem = srv.serialize_pem_with_signer(&ca)?;

// 3. Fingerprint (SHA-256, hex format)
let digest = Sha256::digest(&der);
// "AA:BB:CC:..." → rejestrowany w control plane
```

Biblioteki kryptograficzne (Cargo.toml)

BIBLIOTEKA	ZASTOSOWANIE
`rcgen 0.12`	Generowanie certyfikatow x509 (CA + server) w czystym Rust
`sha2 0.10`	SHA-256 fingerprint certyfikatow serwera
`x25519-dalek 2.0`	Krzywe eliptyczne Curve25519 (klucze WireGuard)
`base64 0.22`	Kodowanie/dekodowanie kluczy i certyfikatow PEM
`nix 0.29`	Sprawdzanie uprawnien root (euid) na Unix
`security-framework`	Keychain macOS — credentials storage
`openssl CLI`	Self-signed cert dla OpenVPN (`req -x509`)

PKI uzywane przez: OpenVPN (CA bundle + fingerprint) i IKEv2 (certyfikaty w /etc/ipsec.d/)

Przeplyw PKI w systemie

pki::ensure_pki()
rcgen: CA + server cert

▶

Env vars
VPN_OVPN_CA_BUNDLE
VPN_OVPN_PEER_FINGERPRINT

▶

register()
CA + fingerprint → control plane

▶

setup_ikev2()
Kopiuje do /etc/ipsec.d/

▶

Desktop
Weryfikuje trust material

Porownanie protokolow i wywolania systemowe

Porownanie protokolow w projekcie

CECHA	WIREGUARD	OPENVPN	IKEV2 / IPSEC
Szyfrowanie	ChaCha20-Poly1305	AES-256-GCM	AES-256-GCM (ECP256)
Wymiana kluczy	Curve25519 (Noise)	RSA 2048 (TLS handshake)	ECDH P-256 (IKEv2 SA)
Auth klienta	Klucz publiczny (wg genkey)	Username/password (verify.sh)	PSK / pubkey / EAP
Interfejs sieciowy	`wg0` (kernel/wireguard-go)	`tun1` (userspace daemon)	kernel IPsec (strongSwan)
Peer management	`wg syncconf` (live, 0 downtime)	`secrets.txt` (no reload)	Statyczna konfiguracja
Serwer start	`ip link add` + `wg setconf`	`openvpn --config --daemon`	`ipsec start --nofork`
Klient (macOS/Linux)	`wg-quick up {conf}`	`openvpn --config --daemon`	`networksetup` / `ipsec up`
Klient (Windows)	`wireguard.exe /installtunnelservice`	`openvpn.exe --config`	`Add-VpnConnection` + `rasdial`
Status	gotowy	gotowy	czesciowy

Sync loop — main.rs (petla synchronizacji peerow)

```rust
// main.rs:468 – Peer Sync Loop (uruchamiany jako tokio::spawn)
let protos_env = env::var("VPN_PROTOCOLS").unwrap_or("wireguard,openvpn,ikev2");
let node = VpnNode::new(&enabled, args.listen_port)?;   // tworzy WireGuardBackend + OpenVpnBackend + IpsecBackend
node.start_all();                                        // start() na kazdym backendzie
setup_nat_and_forwarding()?;                             // sysctl ip_forward=1 + iptables MASQUERADE

loop {
    tokio::time::sleep(Duration::from_secs(2)).await;
    let peers = client.fetch_peers().await?;             // GET /server/peers (Bearer token)
    node.apply_peers(&peers)?;                           // iteruje po backendach: wg syncconf / secrets.txt
}
```

Wykorzystane technologie

KOMPONENT	TECHNOLOGIA	DO CZEGO SLUZY
Frontend	Next.js 15, TypeScript	App Router, SSR, panel uzytkownika i admina, strony marketingowe
	tRPC v11	Typesafe RPC miedzy frontendem a API routes Next.js (8 routerow)
	NextAuth.js, Stripe	Autentykacja (GitHub, Google, magic link) + subskrypcje i checkout
Control Plane	Bun, Fastify	Lekki runtime JS + framework HTTP — REST API (~540 LOC)
	Prisma, PostgreSQL	ORM + baza danych (Neon serverless), wspoldzielony schemat miedzy serwisami
VPN Server	Rust, Tokio, Axum	Async agent VPN — Tokio (I/O, timery), Axum (admin HTTP API)
	x25519-dalek, rcgen, sha2	Kryptografia: klucze WG (Curve25519), certyfikaty x509 (PKI), fingerprint (SHA-256)
Desktop	Tauri, React, Vite	Natywna aplikacja z webowym GUI — lekka alternatywa dla Electron
	Rust Daemon	Osobny proces root — operacje sieciowe, IPC przez Unix Socket (JSON-RPC)
Infra	Pulumi, TypeScript	Infrastructure as Code — VPC, EC2, Lambda, S3, API Gateway, ECR
CI/CD	GitHub Actions	3 workflows: CI (lint+test), desktop build (3 platformy), backend deploy (ECR+Pulumi)

Status realizacji

apps/web — Frontend SaaS	~90%
services/control-plane	~95%
apps/vpn-server — Agent VPN	~85%
apps/desktop — Klient VPN	~80%
services/metrics	~70%
packages/db — Prisma schema	100%
infra/pulumi — IaC	~60%
CI/CD (GitHub Actions)	~90%

Legenda

- Zrobione / blisko konca
- W trakcie

Procenty na podstawie przegladu kodu, nie deklaracji.

Co wplywa na procenty

- **Web 90%** — wszystkie strony dzialaja, proxy i analityka to placeholder
- **Control Plane 95%** — pelne API, brakuje kilku edge case'ow
- **VPN Server 85%** — WG i OpenVPN gotowe, IKEv2 czesciowy (~3 KB vs ~11 KB)
- **Desktop 80%** — GUI i auth ok, integracja daemon ↔ sidecar w toku
- **Metrics 70%** — ingestion dziala, brak API do odczytu metryk
- **Infra 60%** — global stack gotowy, 2 z planowanych regionow skonfigurowane

Frontend i backend — co zrobione

DONE

Frontend SaaS ~90%

- **Auth** — NextAuth.js (GitHub, Google, Email magic link)
- **Stripe** — checkout, subskrypcje (Basic/Pro/Enterprise), billing portal, webhooks
- **Panel uzytkownika** — dashboard, zarzadzanie urzadzeniami, konfiguracje VPN
- **Panel admina** — tokeny, uzytkownicy, provisioning, metryki serwerow
- **tRPC** — 8 routerow (device, billing, admin, servers, account, desktop, analytics, proxies)
- **Email** — Resend (powiadomienia, welcome)

Brakuje: proxy (placeholder), admin analytics (pusta strona)

Control Plane ~95%

ENDPOINT	AUTH	
POST /server/register	Bearer	●
GET /server/peers	Bearer	●
GET /servers, DELETE /servers/:id	API key	●
POST /peer/add, /peers/revoke	API key	●
CRUD /tokens	API key	●

~540 LOC, Zod walidacja, rate limiting, bootstrap tokena z env.

Metrics ~70%

- ● `POST /metrics/vpn` — ingestion metryk z serwerow
- ● `GET /health` — health check
- ● `GET /metrics` — brak API do odczytu

Frontend czyta metryki bezposrednio z bazy przez tRPC.

Desktop — klient VPN

~80%

Zrobione

- **Tauri shell** — natywne okno, buildy macOS/Linux/Windows
- **React GUI** — login, dashboard, wybor serwera, mapa, ustawienia
- **Auth** — email + OTP, deep link (`vpnvpn://`)
- **Daemon client** — IPC przez Unix Socket / Named Pipe
- **WireGuard klient** — keygen, connect, status, disconnect
- **OpenVPN klient** — config gen, auth file, process management
- **IKEv2 klient** — 3 implementacje (macOS/Linux/Windows)
- **Kill switch**, **system tray**, detekcja narzedzi VPN

W trakcie

- **Daemon sidecar** — pobieranie i zarzadzanie binarki daemon
- Testy E2E polaczen na wszystkich platformach
- Auto-update mechanizm

Architektura daemon

```
+------------------+
|   Tauri GUI      |  React + Vite
|   (user space)   |  tRPC client
+--------+---------+
         | IPC (Unix Socket / Named Pipe)
         | JSON-RPC
+--------+---------+
|   Rust Daemon    |  root / admin
|   (privileged)   |  wg-quick, openvpn,
+------------------+  ipsec, networksetup

// Daemon nasluchuje na:
// prod: /var/run/vpnvpn-daemon.sock
// dev:  /tmp/vpnvpn-daemon.sock
```

Infrastruktura i CI/CD

Pulumi IaC ~60%

- **Global stack** (gotowy) — ECR, S3, Lambda + API GW, AMP workspace
- **Regional stacks** — VPC, statyczna pula EC2, Elastic IP, Security Groups
- **Skonfigurowane:** `eu-west-1` `us-east-1`
- Pozostale 8 regionow — kod komponentow gotowy, brak stack files

CI/CD ~90%

- **ci.yml** — lint (Biome), testy, build na push/PR
- **desktop-build.yml** — macOS arm64/x64, Linux, Windows + S3 upload
- **deploy-backend.yml** — ECR push, Pulumi deploy, path-based triggers
- Srodowiska: `staging` `production`

Pipeline deploy

Push (main / staging) → **Lint + Test** (Biome, Jest) → **Build** (Docker, Rust, Tauri) → **Push artefakty** (ECR + S3) → **Deploy** (Pulumi + Lambda)

Siec na serwerze VPN (main.rs:100)

```
// setup_nat_and_forwarding() - wywolywane po node.start_all()
sysctl -w net.ipv4.ip_forward=1                          // wlaczenie forwarding pakietow
ip route show default | grep "dev"                        // detekcja interfejsu (eth0/en0)
iptables -t nat -C POSTROUTING -o eth0 -j MASQUERADE      // sprawdz czy regula istnieje
iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE      // NAT dla ruchu VPN → internet
// Interfejsy VPN tworzene przez backendy: wg0 (WireGuard), tun1 (OpenVPN), ipsec (IKEv2)
```

Co zostalo do zrobienia

Priorytet

- Dokonczenie integracji **daemon ↔ GUI** (sidecar download)
- **IKEv2 backend** — dokonczenie implementacji, testy
- **Metrics read API** — endpointy do odczytu metryk
- Konfiguracja **pozostalych 8 regionow** AWS
- **Testy E2E** produkcja: signup → platnosc → VPN
- Admin **analytics** — wizualizacja danych

Backlog

- **Proxy** — obsluga SOCKS5 / HTTP
- Deploy wezlow VPN **z panelu admina**
- **Auto-update** desktopa
- Testy obciazeniowe serwerow VPN
- Metryki per-user (transfer)

Zrobione ostatnio

- Dodanie **IKEv2** z generowaniem PKI
- **Username/password** auth (OVPN, IKEv2)
- Zamiana ASG na **statyczna pule EC2**
- Refaktor desktop na **tRPC client**
- Rollback interfejsu przy bledzie **WG handshake**
- **Dockerfile.local** — testy integracyjne
- Usuniecie legacy envs

~83%	6	5	14	3
szacowany postep calkowity	zadania priorytetowe	backlog	modeli w bazie (Prisma)	protokoly VPN

Dziekuje za uwage

Pytania?

Projekt i implementacja platformy do zarzadzania usluga VPN